



BOOTCAMP FRONTEND & REACT · SEMANA 4

La tienda terminada

Clase 11 — último día de JavaScript: los 38 productos en paralelo, el plan B, los filtros, el buscador, el carrito de verdad y el checkout.

4 h 30 · L·M·V

Proyecto TechCart

STEERCLV

PRIMERO LA DEUDA DE LA CLASE 10 · DESPUÉS, LO QUE FALTA PARA SER UNA TIENDA

Agenda

1. Dónde quedamos de verdad

16'

repaso de la Clase 10

2. Promise.all

37'

los 38 productos, en paralelo

3. El plan B

28'

respaldo + el aviso honesto

4. Filtro por categoría

40'

el estado derivado

5. El buscador

34'

y el peligro de innerHTML

6. El carrito con cantidades

50'

el carrito de verdad

7. Remates + el checkout

44'

la compra que funciona

8. Cierre + 5 tareas

21'

y el anticipo a TypeScript

**Receso de 15 min** entre el bloque 3 y el 4 · naranja = **deuda de la Clase 10** · verde = **lo que nunca se dictó**

Cuatro por el chat

¿Qué devuelve `fetch`?

Si el servidor responde 404, ¿se dispara el `catch`?

¿Para qué sirve el adaptador?

¿Cuáles son los tres estados al pedir datos?

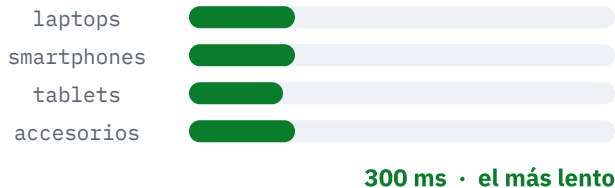
CUATRO CATEGORÍAS · CUATRO DIRECCIONES · NINGUNA ESPERA A LA OTRA

Cuatro pedidos, uno al lado del otro

✗ CUATRO AWAIT SEGUIDOS



✓ PROMISE.ALL



```
const CATEGORIAS = ["laptops", "smartphones",  
  "tablets", "mobile-accessories"]  
  
const listas = await Promise.all(  
  CATEGORIAS.map(pedirCategoria)  
)  
  
return listas.flat() // 38 productos
```

map devuelve **cuatro promesas ya viajando**: `Promise.all` no las dispara, solo las **espera**. Y devuelve los cuatro resultados **en el orden en que se pidieron**.



Caso real: la escalera se llama **waterfall**. El clásico: un bucle sobre 50 productos con un `await` adentro = 50 peticiones en fila. **Un `await` dentro de un bucle:** sospechen.

DEPENDEMOS DE UN SERVIDOR QUE NO CONTROLAMOS

Si la API se cae, la tienda **abre igual**

```
import { productos as respaldo } from
"./datos.js"

try {
  const listas = await Promise.all(...)
  return { productos: listas.flat(), esRespaldo:
false }
} catch (error) {
  console.warn("API caída, uso el respaldo")
  return { productos: respaldo, esRespaldo: true
}
}
```

Los cinco productos que íbamos a tirar **no se tiran**: son el plan B. Y el `try/catch` va en `api.js`, no en `main.js`: quien sabe qué hacer cuando los datos no llegan es la **capa de datos**.

esRespaldo: true

...y el usuario **no se entera**. Cree que la tienda tiene cinco productos. Eso es mentirle.

aviso.hidden = !esRespaldo

Una línea, los **dos** casos. No hay un `if` que muestra y otro que esconde: hay un **dato** y la pantalla lo refleja.

 **Degradación elegante**: Spotify te muestra la biblioteca sin conexión; el correo te deja leer lo ya bajado. Cuando algo falla, no se cae todo — se cae **menos**.

ESTÁN EN EL HTML DESDE LA CLASE 2 Y NUNCA HICIERON NADA

La **etiqueta** es para la persona. El **valor**, para la máquina.

```
<!-- el usuario lee "Audio"... -->
<a href="#" data-categoria="mobile-
accessories">Audio</a>
// ...y el código usa "mobile-accessories"

listaCategorias.addEventListener("click", (evento)
=> {
  const enlace = evento.target.closest("a[data-
categoria]")
  if (!enlace) return
  evento.preventDefault() // el <a> quiere
navegar: se lo prohíbo
  categoriaActiva = enlace.dataset.categoria
  pintarCatalogo()
})
```

El día que marketing quiera que diga "Accesorios y audio", se cambia el texto y el código no se enteró.

Nunca decidan la lógica leyendo el texto de la pantalla.

💡 **¿Y por qué no quitarle el href?** Porque un <a> sin href deja de ser un enlace: no se alcanza con el **teclado**, no lo anuncia un lector de pantalla y no muestra el cursor de mano. Se mantiene el href y se cancela el comportamiento. **Por eso existe preventDefault.**

Lo que se ve **no se guarda: se calcula**



Tres **datos** son la única verdad. Lo que se ve **no es un cuarto dato**: es una **consecuencia**. Por eso `productosVisibles` es una **función** y no una variable.

Un dato guardado hay que **mantenerlo al día**; un dato calculado **no puede estar desactualizado nunca**. Con estado derivado, ese bug es **imposible de escribir**.

 **Caso real:** el filtro dice "Tablets", la grilla muestra tablets... y el textito de arriba dice **"38 resultados"**. Dos lugares calculando por su lado.

UN DATO MÁS · Y LOS DOS FILTROS SE COMBINAN SOLOS

El evento **input**: en cada tecla

```

buscador.addEventListener("input", (evento) => {
  termino = evento.target.value
  pintarCatalogo()
})

// los dos filtros, en el ÚNICO lugar que decide
qué se ve
const productosVisibles = () => {
  const porCategoria = categoriaActiva === "todas"
    ? productos
    : productos.filter(p => p.categoria ===
      categoriaActiva)

  const buscado = termino.trim().toLowerCase()
  if (buscado === "") return porCategoria
  return porCategoria.filter(p =>
    p.nombre.toLowerCase().includes(buscado))
}

```

Ni una línea que diga "si hay categoría **y además** hay búsqueda". Agregué un dato y unas líneas en la misma función. **Ese** era el pago del estado derivado.

input ≠ **keydown**

input también se dispara al **pegar** texto y al **dictar por voz**.

trim · **toLowerCase** · **includes**

un espacio suelto no vacía el catálogo; "MACBOOK" encuentra "macbook"; y busca en **cualquier parte** del nombre.

no le pide nada a la API

filtra los 38 que ya están en memoria: por eso puede reaccionar en cada tecla. Si preguntara al servidor harían falta **debounce** y **AbortController**.

LA VULNERABILIDAD MÁS COMÚN DE LA WEB

Si el texto vino de un **usuario**, nunca va crudo en **innerHTML**

...y esto escribe el usuario en el buscador:

```
<img src=x
onerror="document.body.style.background='red'">
```

Yo puse un color porque es visual. Con la misma línea se lee el localStorage, se roba la sesión o se manda el carrito a otro servidor.

La cura, en tres líneas:

```
export const escaparTexto = (texto) => {
  const caja = document.createElement("div")
  caja.textContent = texto // nunca interpreta
  HTML
  return caja.innerHTML // < se volvió &lt;
}
```

textContent

pone **texto** y es seguro. Para el título, el total, el resumen.

innerHTML

interpreta **etiquetas** y es peligroso. Solo para HTML que **ustedes** armaron.

 **XSS almacenado** es el caro: si los **vendedores** escriben el nombre del producto, le pasa a **todos** los visitantes, no solo al que escribió. La versión clásica: la sección de comentarios con un `<script>` adentro.

 En **React** esto **no se puede escribir**: JSX escapa todo lo que interpolan. Si de verdad querés HTML crudo, la función se llama `dangerouslySetInnerHTML` — con esa palabra adentro, a propósito.

NINGUNA TIENDA DEL MUNDO HACE LO DE LA IZQUIERDA

Un ítem ya no es un producto: es un producto con una cantidad

✗ HOY · TRES CLICS EN LA MISMA LAPTOP

MacBook Pro 14	S/ 1999.99
MacBook Pro 14	S/ 1999.99
MacBook Pro 14	S/ 1999.99

✓ LO QUE HACEMOS HOY

MacBook Pro 14 - × 3 + S/ 5999.97

```
// ya estaba: copia ESE item con su cantidad + 1
return items.map(item =>
  item.id === producto.id
    ? { ...item, cantidad: item.cantidad + 1 }
    : item
)
```

El patrón de la inmutabilidad, y en React lo van a escribir cien veces: recorro la lista y devuelvo **una copia modificada** o **el mismo ítem tal cual**. Sin push, sin asignar a una propiedad, sin for.

💡 `existente.cantidad++` funcionaría hoy... y **no** funciona en React: no repinta, porque el array sigue siendo el mismo objeto. Escribimos hoy la costumbre que allá es obligatoria.

Un solo listener, **tres botones**

```
listaCarrito.addEventListener("click", (evento) => {
  const boton = evento.target.closest("button[data-accion]")
  if (!boton) return

  const id = Number(boton.dataset.id) // dataset SIEMPRE es texto
  const accion = boton.dataset.accion

  if (accion === "mas") carrito =
    cambiarCantidad(carrito, id, 1)
  if (accion === "menos") carrito =
    cambiarCantidad(carrito, id, -1)
  if (accion === "quitar") carrito =
    quitarItem(carrito, id)

  guardarCarrito(carrito); pintarCarrito()
})
```

El closest ya no pregunta por una acción concreta: pregunta por **"cualquier botón que tenga una acción"**. Después lee **cuál** y decide. El día que agreguen "Guardar para después": una fila en el HTML y un if acá. **No un listener nuevo.**

bajar a 0 = quitar

un.filter(item => item.cantidad > 0) al final, y la regla vive en **un solo lugar**.

volvió el id

ayer no se podía quitar por id porque había filas repetidas. No arreglamos el "quitar": arreglamos el **modelo de datos**, y el "quitar" se arregló solo.

REFRESQUEN LA PÁGINA · Y NADIE AVISA



NaN productos · S/ NaN

```
// lo que guardó la versión de hace 20 minutos:
{ id: 1, nombre: "MacBook Pro 14", precio:
1999.99 }
// ...sin cantidad

undefined * 1999.99 → NaN
0 + NaN → NaN // el veneno se propaga
```

Cura 1 – sanear en la puerta:

```
// en cargarCarrito, al leer del localStorage
return items.map(item =>
  ({ ...item, cantidad: item.cantidad ?? 1 })))
```

 **Migración de datos**, y es uno de los bugs más caros que existen. Sale una versión nueva y los usuarios **viejos** —que ya tienen datos guardados con la forma anterior— abren la app y les explota. Los nuevos no ven nada raro. **En su máquina no se reproduce**, porque usted probó con el almacenamiento limpio.

Cura 2 — versionar la clave: "techcart_carrito_v2". Los datos viejos quedan huérfanos y el programa arranca limpio. Más brusco, pero honesto y sin código de conversión.

tres puertas, tres aduanas

La API puede no traer brand. El localStorage puede tener la forma de otra versión. El usuario puede escribir un . **Lo que entra de afuera nunca es de fiar.**

LAS DOS QUE FALTABAN DE LA FAMILIA · DEVUELVEN TRUE O FALSE

¿Hay al menos uno? ¿Cumplen todos?

filter

devuelve una **lista** con los que cumplen

some

¿hay **al menos uno**? → true / false

every

¿cumplen **todos**? → true / false

JAVA · STREAMS

```
anyMatch(...)  
allMatch(...)
```



TEHCART · EL BOTÓN DE CONFIRMAR

```
const hayProblemas = carrito.some(  
  item => item.cantidad > item.stock)  
  
botonConfirmar.disabled =  
  carrito.length === 0 || hayProblemas
```

some para buscar problemas, **every** para confirmar que todo está bien. Elijan la que hace la línea legible. · Y ojo: `[] .every (...)` en un array **vacío** devuelve **true**.

EL FORMULARIO ESTÁ EN LA PÁGINA DESDE LA CLASE 1 · HOY FUNCIONA

La validación que **no escribimos** ya está trabajando

1

submit

+
`preventDefault():`
si no, la página
recarga y se pierde
todo

2

¿es válido?

el navegador ya frenó
lo inválido: el evento
**solo se dispara si
pasa**

3

leer los 9 campos

`Object.fromEntries(new
FormData(f))`

4

guardar el pedido

clave **aparte:**
`techcart_pedidos`

5

vaciar y avisar

y la pantalla entera se
acomoda sola

LO QUE REGALÓ LA CLASE 1

Los `required`, el `type="email"`, el `minlength` y el
`pattern` del teléfono. **Cero líneas de validación
escritas hoy**, mensajes en español y accesibles.



PERO NO ES SEGURIDAD

Cualquiera abre DevTools, borra el `required` y manda
el formulario vacío. **Validar en el cliente para ayudar;
validar en el servidor porque es el único lugar donde
el usuario no manda.**

Y el orden importa: **primero se guarda el pedido, después se vacía el carrito.** Al revés, un fallo en medio deja al usuario sin las
dos cosas.

HACE TRES SEMANAS ERA UN ARCHIVO HTML VACÍO

Seis archivos, un trabajo cada uno



Cuando algo se rompa, ya saben en cuál mirar — y **eso**, no la sintaxis, es lo que separa un programa de un montón de código.

Resumen

C7 · map/filter/reduce

transformar, elegir y resumir sin un solo for

C8 · ES6+ y módulos

desestructurar, spread, inmutabilidad, un archivo un trabajo

C9 · DOM y eventos

delegación, dataset, la página deja de ser una foto

C10 · datos reales

fetch, promesas, adaptador y los tres estados

C11 · estado derivado

los datos, y una función que calcula lo que se ve

C11 · el MVP

paralelo, plan B, buscador, cantidades y checkout

Si tuviera que dejarles **una** idea de las cinco clases: **hay datos, y hay funciones que dibujan esos datos.**

Un evento nunca toca la pantalla: **cambia un dato y manda a pintar.**

5 tareas

FÁCIL

Envío gratis desde \$50: que `resumenCarrito` devuelva `envioGratis` y el carrito lo muestre. Una comparación y un ternario.

INTERMEDIO

El número por categoría: "Laptops (5)". Un `reduce` cuyo acumulador es un **objeto** — el más útil que existe.

INTERMEDIO

Ordenar el catálogo: un `<select>` con el evento `change`. Es un **cuarto dato** en `productosVisibles`. Y `sort` muta: `[...lista].sort()`.

DIFÍCIL

El detalle del producto: al clic en la tarjeta, `fetch` a `/products/{id}` — la descripción no viene en el listado. Con sus estados de carga y error.

DIFÍCIL

"Mis pedidos": leer `techcart_pedidos` y pintarlo con `<details>`. Ojo: el nombre del cliente lo escribió un usuario → `escaparTexto`.

Próxima clase

¿Dónde está escrito **qué es un producto?**

Hoy pasó **tres veces** lo mismo y nadie avisó: el respaldo mostró **undefined** en la marca, el carrito guardado dijo **S/ NaN**, y el buscador se habría roto al escribir la primera letra si marca viniera vacía. Los tres son el **mismo** error: un objeto que no tenía la forma que el código esperaba.

Y la respuesta a la pregunta es incómoda: está en `mapearProducto`, en `datos.js`, en la desestructuración de `tarjetaProducto`, en los campos del buscador... **en cuatro lugares y en mi cabeza**. Hoy me acordé de agregar `marca` al respaldo. La próxima vez no me voy a acordar.

Se escribe una vez, y el editor avisa antes de correr. Se llama TypeScript — y para ustedes, que vienen de Java, va a ser como volver a casa.